

9주차

Vision Transformer (ViT)의 등장 배경

- ViT(Vision Transformer) 는 2020년 구글의 논문
"An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale" 에서 처음 제안된 모델
- 이미지 인식을 위해 자연어 처리에서 사용되던 Transformer 구조를 이미지 분류에 적용한 모델
- 기존의 CNN과 달리 합성곱(Convolution) 을 사용하지 않고, 이미지를 작은 패치(Patch) 로 나눈 뒤 셀프 어텐션(Self-Attention) 으로 처리
- CNN은 지역적(Local) 특징 추출에 강점이 있으나, ViT는 전체 이미지를 한 번에 처리하면서 전역적(Global) 관계를 학습

ViT와 Swin Transformer, CvT의 차이

- ViT는 패치가 순차적으로 입력되어 2차원 이미지 구조를 충분히 반영하지 못하는 한계가 있다.

이를 해결하기 위해 Swin Transformer 와 CvT (Convolutional Vision Transformer) 가 제안됨.

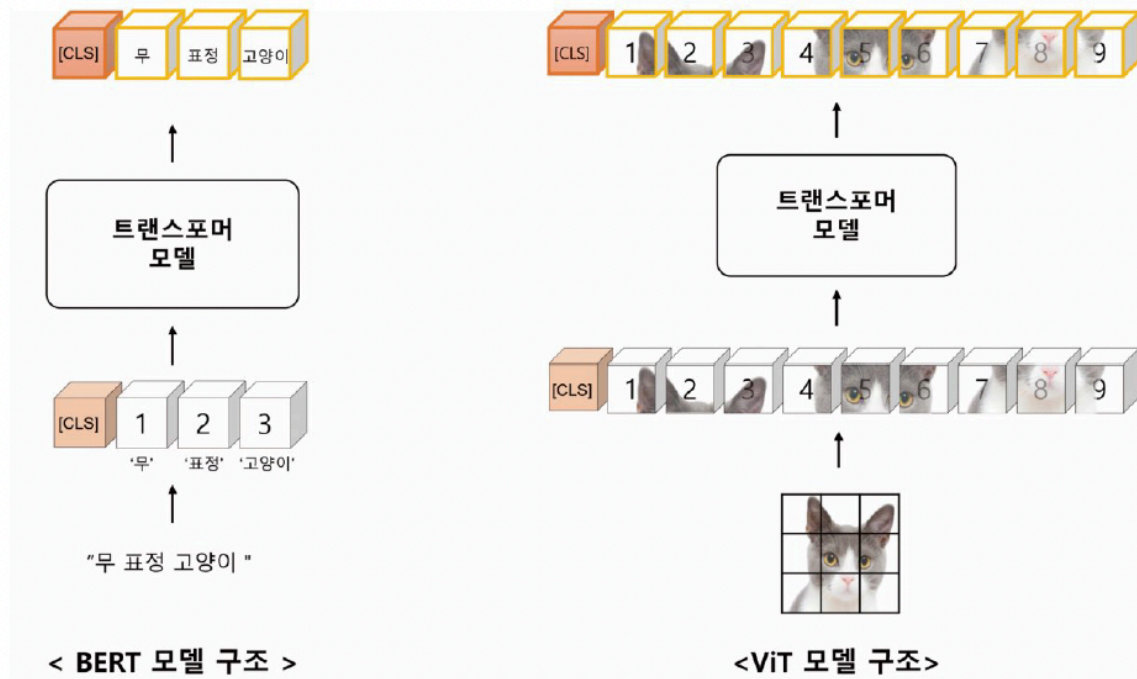
(1) Swin Transformer

- 이미지를 로컬 윈도우(Local Window) 단위로 나누어 어텐션 계산을 수행.
- 지역 특징을 학습하면서도 여러 계층에서 점진적으로 특징을 통합.
- "Shifted Window" 기법을 통해 윈도우 간 정보 교환도 가능.

(2) CvT (Convolutional Vision Transformer)

- ViT에 합성곱 계층을 결합하여 저수준(로컬) 특징과 고수준(전역) 특징을 동시에 학습.
- Query, Key, Value 계산 시 합성곱 기반의 지역 정보를 활용해 계산량 감소.

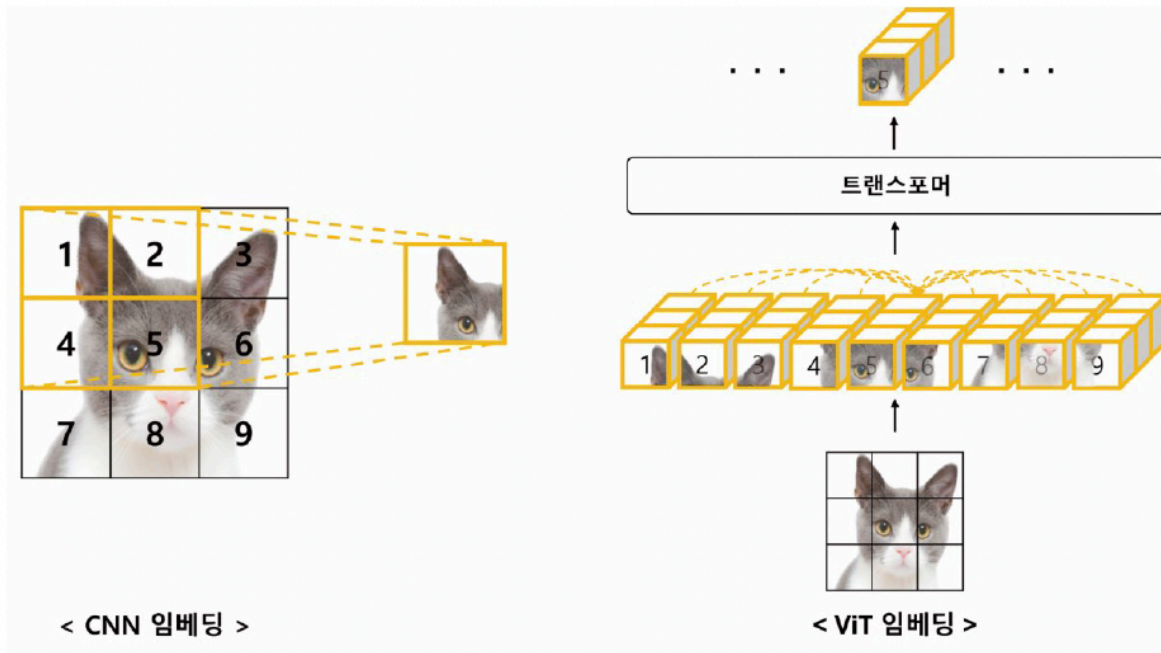
ViT와 BERT 모델의 구조 비교



모델	입력 단위	처리 방식
BERT	문장의 토큰(단어 단위)	텍스트를 순차적으로 입력
ViT	이미지 패치(조각 단위)	이미지를 작은 패치로 나누어 순서대로 입력

- ViT는 이미지를 패치 단위로 쪼개어 각 패치를 "토큰(token)"처럼 처리한다.
- 예: 고양이 이미지를 9개 패치로 분할 → 각 패치를 Transformer 입력으로 사용.
- 이렇게 ViT는 시각 데이터를 자연어 시퀀스처럼 변환하여 학습한다.

CNN vs ViT 임베딩 방식 비교



(1) CNN

- 작은 필터를 사용해 국소 영역(Receptive Field) 만 보고 학습.
- 한 번에 전체 이미지 특징을 보려면 여러 계층이 필요.
- 예를 들어 고양이의 한쪽 눈만 학습하고, 다른 부분은 고려하지 않음.

(2) ViT

- 이미지를 패치 단위로 나누어 모든 패치 간의 관계를 동시에 학습(Self-Attention)
- 각 패치가 서로의 관계를 고려하므로 전체 이미지의 전역적 특징을 한 번에 학습.
- 즉, ViT는 더 넓은 "Attention Distance"를 갖는다.

ViT의 장단점

장점

- 이미지 전체를 한 번에 처리 → 전역적 정보 학습에 강함.
- 패치 단위로 처리 → 모델 크기를 조정해 다양한 해상도 처리 가능.
- 합성곱 없이도 높은 성능.

단점

- 입력 이미지 크기가 고정되어야 함 (resize 필요).

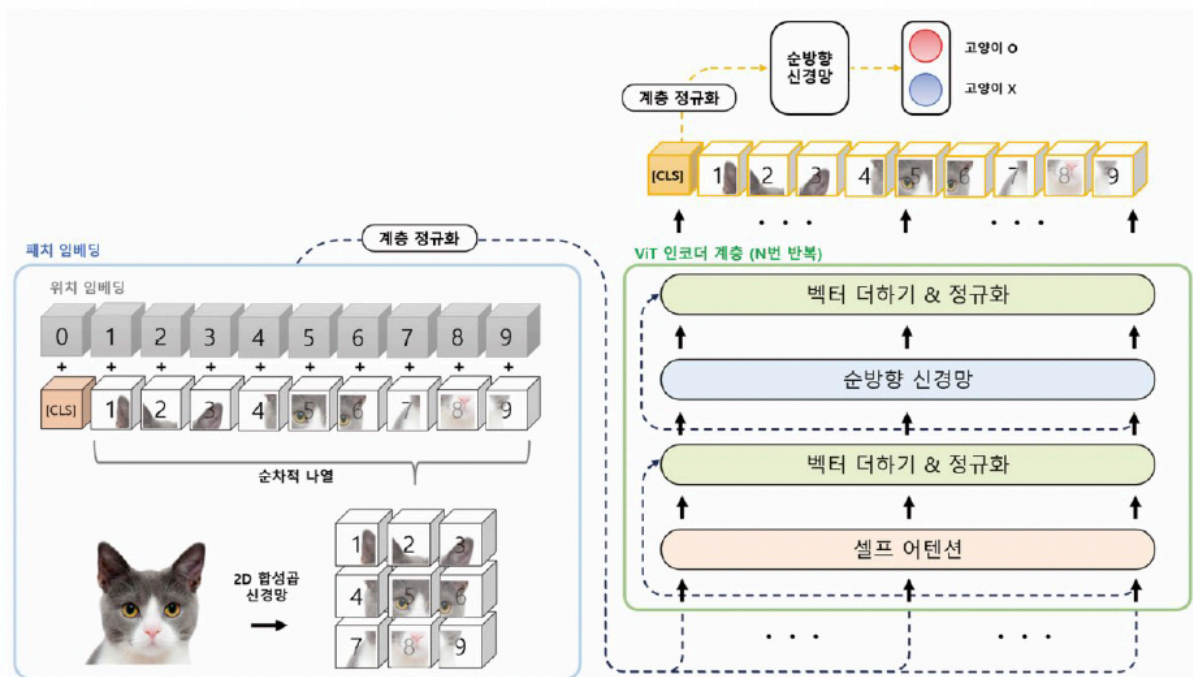
- CNN에 비해 공간적 위치 정보를 직접적으로 반영하지 못함 → 위치 임베딩(Position Embedding) 필요.
- 데이터량이 적으면 성능 저하가 큼 (대규모 데이터에 적합).

귀납적 편향(Inductive Bias)

모델	귀납적 편향 종류	설명
CNN	지역적 편향	이미지의 공간적 근접 관계에 집중
RNN	순차적 편향	시간 순서 관계를 잘 표현
ViT	거의 없음	데이터에서 직접 관계를 학습함

- CNN은 지역적(Local) 관계를,
RNN은 시간적(Sequential) 관계를 가정하는 반면,
ViT는 특정 편향이 거의 없어 데이터로부터 관계를 직접 학습한다.
- 따라서 ViT는 대규모 데이터에서는 강력하지만, 작은 데이터에서는 과적합(overfitting) 위험이 있다.

ViT 모델 구조



1. 패치 임베딩 (Patch Embedding)

- 이미지를 작은 패치 단위로 나누고 각 패치를 벡터로 변환.
- CNN의 합성곱과 유사하게 커널 크기(Kernel Size)와 간격(Strde)을 하이퍼파라미터로 사용.
- 예: 이미지 9×9, 커널 3×3, stride=3 → 총 9개의 패치 생성.

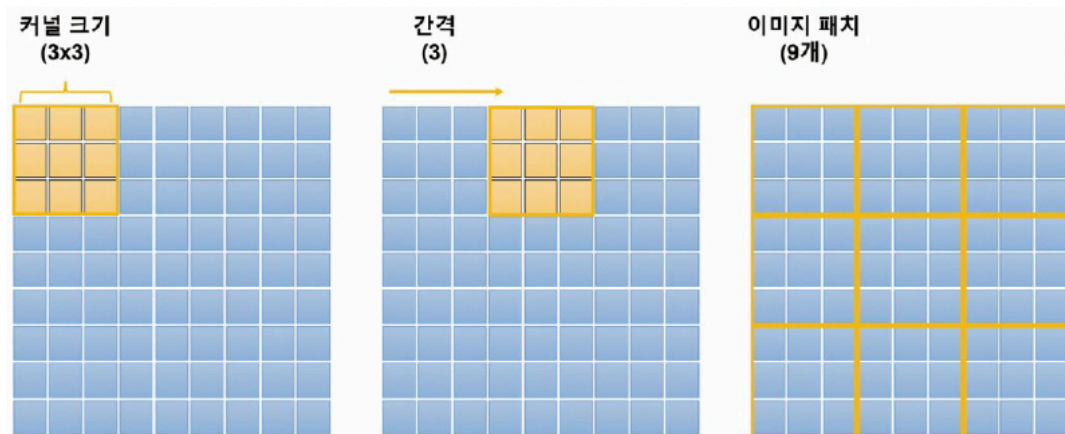


그림 10.5 이미지 패치 생성 과정

2. 위치 임베딩 (Position Embedding)

- 패치 순서에는 위치 정보가 없기 때문에 위치 임베딩을 추가하여 공간 정보 보존.

3. 인코더 계층 (Encoder Layers)

- 여러 층의 Transformer 인코더 반복.
- 각 층은 Self-Attention 과 Feed-Forward Network 로 구성.
- [CLS] 토큰은 전체 이미지의 대표 벡터로 사용되어 분류에 활용됨.

4. 출력 단계 (Classification Head)

- [CLS] 토큰 벡터를 완전연결층(Dense Layer)에 통과시켜 최종 클래스(예: 고양이, 개 등) 예측.

FashionMNIST 실습

- 데이터셋: 의류 이미지 10종류 (T-shirt, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, Ankle Boot)
- 학습용 60,000장 / 테스트용 10,000장
- 이번 실습에서는 간단히 학습 데이터 10,000장, 테스트 1,000장만 샘플링하여 사용.

데이터 불러오기

```

from torchvision import datasets
from torch.utils.data import Subset
from itertools import chain
from collections import defaultdict

def subset_sampler(dataset, classes, max_len):
    # 각 클래스별 인덱스 저장
    target_idx = defaultdict(list)
    for idx, label in enumerate(dataset.train_labels):
        target_idx[int(label)].append(idx)

    # 클래스별 최대 max_len개 샘플 선택
    indices = list(chain.from_iterable(
        [target_idx[i][:max_len] for i in range(len(classes))]))
    ))
    return Subset(dataset, indices)

# 데이터 다운로드 및 불러오기
train_dataset = datasets.FashionMNIST(root="../datasets", download=True,
train=True)
test_dataset = datasets.FashionMNIST(root="../datasets", download=True, t
rain=False)

subset_train_dataset = subset_sampler(train_dataset, train_dataset.classes,
1000)
subset_test_dataset = subset_sampler(test_dataset, test_dataset.classes, 1
00)

```

Swin Transformer

- 등장 이유: ViT의 지역 정보 손실 문제를 해결하기 위해 제안됨.
- 핵심 아이디어:
 - 이미지를 로컬 윈도우(Local Window) 단위로 나눠 윈도우 내 어텐션(W-MSA) 수행.
 - 다음 레이어에서는 윈도우를 **살짝 이동(Shifted Window)**시켜 SW-MSA 적용 → 윈도우 간 정보 교환 가능.

- 효과: 지역적(로컬) 특성과 전역적(글로벌) 특성을 모두 학습할 수 있음.

상대적 위치 편향(Relative Position Bias)

- 어텐션에서 단순히 Query-Key 내적만 사용하는 대신,
위치 차이(상대적 좌표)를 고려해 가중치를 부여하는 기법.
- 식:
[
$$\text{Attention}(Q, K, V) = \text{Softmax}(QK^T / \sqrt{d} + B)V$$

]
여기서 (B)는 상대적 위치 임베딩(Relative Position Embedding)
- 코드 설명
 - relative_coords로 각 패치 간 x, y 위치 차 계산
 - window_size로 좌표 조정 후 relative_position_index 생성
 - relative_position_bias_table에서 위치별 bias 값을 가져옴
 - 최종적으로 어텐션 스코어 계산 시 $QK^T + B$ 형태로 반영

패치 임베딩 (Patch Embedding)

- 이미지를 **작은 패치(예: 16×16)**로 잘라서 벡터로 변환하는 과정.
- CNN의 Conv2D(kernel_size=16, stride=16) 연산으로 구현됨.
- 예:
224×224 이미지를 16×16으로 자르면 14×14 = 196개의 패치가 생성됨.
각 패치는 768차원으로 변환됨 → [196, 768]
- 앞에 [CLS] 토큰을 추가해 전체 이미지의 대표 벡터 역할 수행 → [197, 768]

위치 임베딩 (Position Embedding)

- 트랜스포머는 순서 정보를 인식하지 못하므로, 각 패치의 위치를 인코딩함.
- ViT에서는 **절대적 위치 임베딩(absolute)**을 사용하거나,
Swin에서는 **상대적 위치 편향(relative)**을 사용.

인코더 계층 구조

- ViT 인코더는 트랜스포머의 멀티 헤드 셀프 어텐션(MSA) + **피드포워드 신경망 (FFN)**으로 구성.
- N개의 인코더 블록을 반복 적용하여 패치 간의 관계를 학습.
- 최종적으로 [CLS] 토큰 벡터를 이용해 전체 이미지의 분류를 수행.

이미지 전처리 (AutoImageProcessor)

- 사전 학습된 ViT 모델(google/vit-base-patch16-224-in21k)의 설정을 불러와 동일한 전처리 적용.
- 주요 단계:
 1. ToTensor() → 이미지 → Tensor 변환
 2. Resize() → (224, 224)로 크기 조정
 3. Lambda() → 흑백 이미지 → 3채널 복제
 4. Normalize() → 평균/표준편차로 정규화

데이터 로더 구성

- PyTorch DataLoader를 사용해 배치 단위로 데이터 불러오기.
- collator() 함수에서 이미지와 라벨을 묶어 {"pixel_values": ..., "labels": ...} 구조로 반환.
- pixel_values: 이미지 Tensor (배치 크기, 채널 수, 높이, 너비)
- labels: 정답 라벨 Tensor (배치 크기)

ViT 모델 로드

- Hugging Face의 ViTForImageClassification 클래스로 사전 학습된 모델 불러오기.
- num_labels, id2label, label2id 설정으로 새 데이터셋에 맞게 미세 조정.
- 출력층(classifier)은 [Linear(in_features=768, out_features=10)]
→ 10개의 패션 클래스 예측.

패치 임베딩 확인

- model.vit.embeddings.patch_embeddings
: Conv2D(16×16, stride=16)으로 패치 생성 확인 가능.

- 출력 형태:

```
patch embeddings shape: [32, 196, 768]
[CLS] + patch embeddings shape: [32, 197, 768]
```

하이퍼파라미터 설정 (TrainingArguments)

- 모델 학습 환경 설정:

```
output_dir="../models/ViT-FashionMNIST"
learning_rate=1e-5
batch_size=16
num_train_epochs=3
metric_for_best_model="f1"
weight_decay=0.001
```

- 매개변수 의미:
 - save_strategy: 체크포인트 저장 주기
 - evaluation_strategy: 평가 주기
 - logging_steps: 로그 출력 간격
 - load_best_model_at_end: 가장 성능 좋은 모델 저장
 - metric_for_best_model: 모델 평가 기준 (정확도, F1 등)

평가 지표 설정 (Macro F1 Score)

- 다중 클래스 분류에서 각 클래스의 F1 점수 평균을 계산.
- 함수 구조:

```
predictions, labels = eval_pred
predictions = np.argmax(predictions, axis=1)
macro_f1 = metric.compute(predictions=predictions, references=labels,
                           average="macro")
```

- 모든 클래스의 성능을 고르게 평가 가능.

최종 학습

- Trainer 클래스를 사용해 전체 파이프라인 통합:

```
trainer = Trainer(
    model=model,
    args=args,
    train_dataset=subset_train_dataset,
    eval_dataset=subset_test_dataset,
    compute_metrics=compute_metrics,
    data_collator=lambda x: collator(x, transform)
)
trainer.train()
```

- 자동으로 데이터 로딩, 전처리, 모델 학습, 평가, 체크포인트 저장까지 수행.

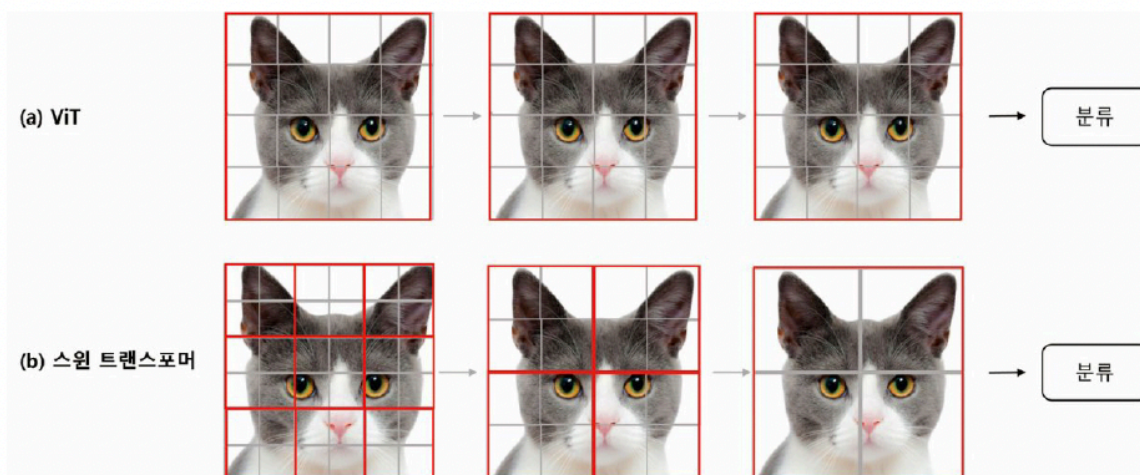


그림 10.6 ViT와 스윈 트랜스포머의 네트워크 구조

Swin Transformer

(1) 등장 배경

- ViT는 고정된 패치 크기, 높은 계산량, 지역 정보 손실 등의 문제 존재
- 이를 해결하기 위해 2021년 Microsoft에서 Swin Transformer 제안

Swin Transformer 구조

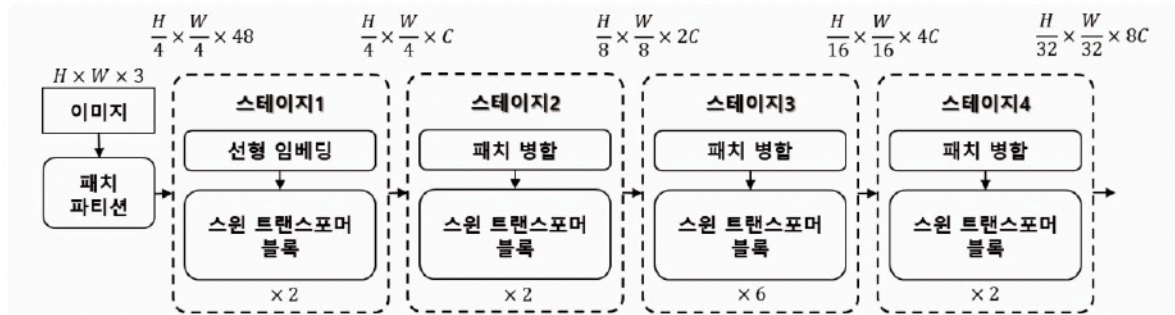


그림 10.7 스윈 트랜스포머 구조

(1) 핵심 개념

- 입력 이미지를 로컬 윈도우(Local Window) 단위로 처리
- 이후 Shifted Window(순환 시프트)로 윈도우를 이동 → 윈도우 간 관계 학습

(2) 계층적 구조 (Hierarchical Design)

- Swin은 여러 스테이지(Stage 1~4)로 구성
- 각 스테이지마다 해상도를 절반으로 줄이고 채널 수는 2배 증가
- 따라서 **다양한 스케일의 특징(멀티 스케일 특징)**을 추출 가능

(3) 구성 요소

구성요소	설명
패치 파티션 (Patch Partition)	이미지를 작은 패치로 분할
선형 임베딩 (Linear Embedding)	각 패치를 벡터로 변환
Swin Transformer Block	W-MSA와 SW-MSA로 구성된 핵심 블록
패치 병합 (Patch Merging)	해상도 감소 & 채널 수 증가

Swin Transformer의 핵심 연산

(1) W-MSA (Window-based Multi-Head Self-Attention)

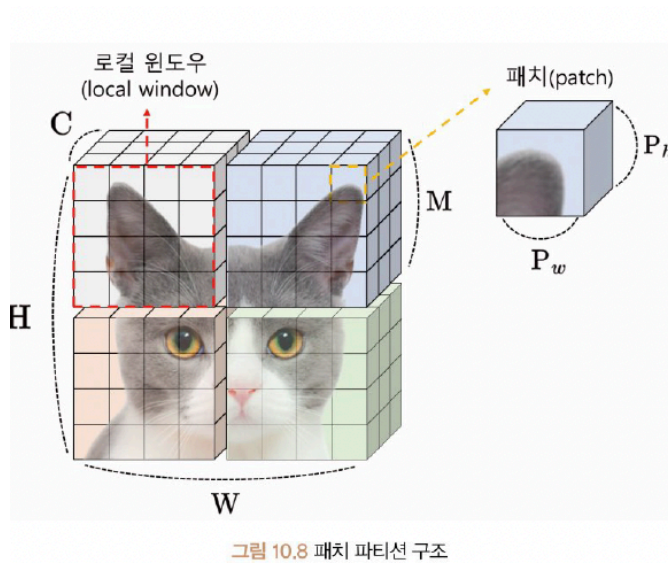
- 하나의 윈도우(예: 7×7) 내에서 패치들 간의 Self-Attention 수행
- 계산량 줄고, 지역적 특징 학습에 강함
- 하지만 윈도우 간 연결 정보는 없음

(2) SW-MSA (Shifted Window MSA)

- 윈도우를 반 칸(Shift) 이동시켜 인접 윈도우끼리 정보 교환 가능
- 윈도우 간 경계 정보도 학습
- Attention Mask를 통해 불필요한 계산 방지

Swin Transformer의 단계별 구성

(1) Stage 1

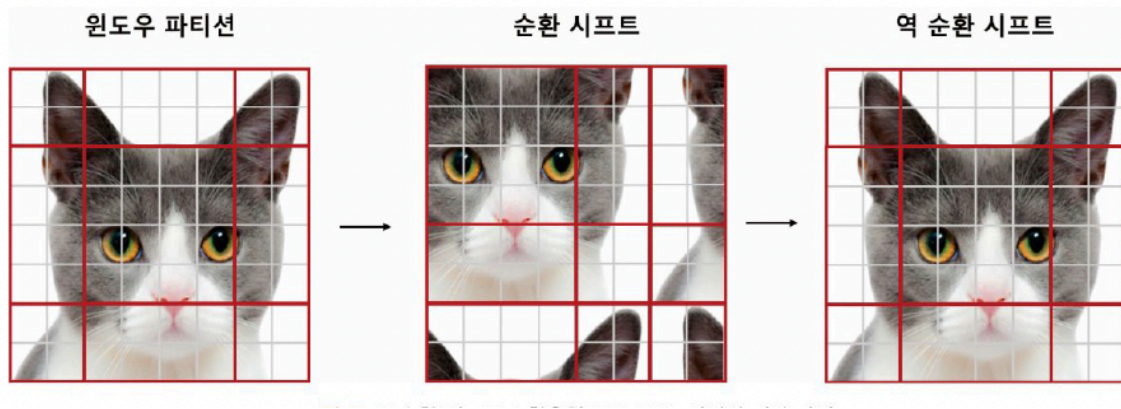


- 입력: $[H, W, 3]$ 이미지
- Patch Partition 후 Linear Embedding
- W-MSA + SW-MSA 블록 2개

(2) Stage 2~4

- Patch Merging으로 해상도 절반, 채널 2배씩 증가
- 예:
 - Stage 1: $[H/4, W/4, 48]$
 - Stage 2: $[H/8, W/8, 96]$
 - Stage 3: $[H/16, W/16, 192]$
 - Stage 4: $[H/32, W/32, 384]$

윈도우와 순환 시프트의 역할

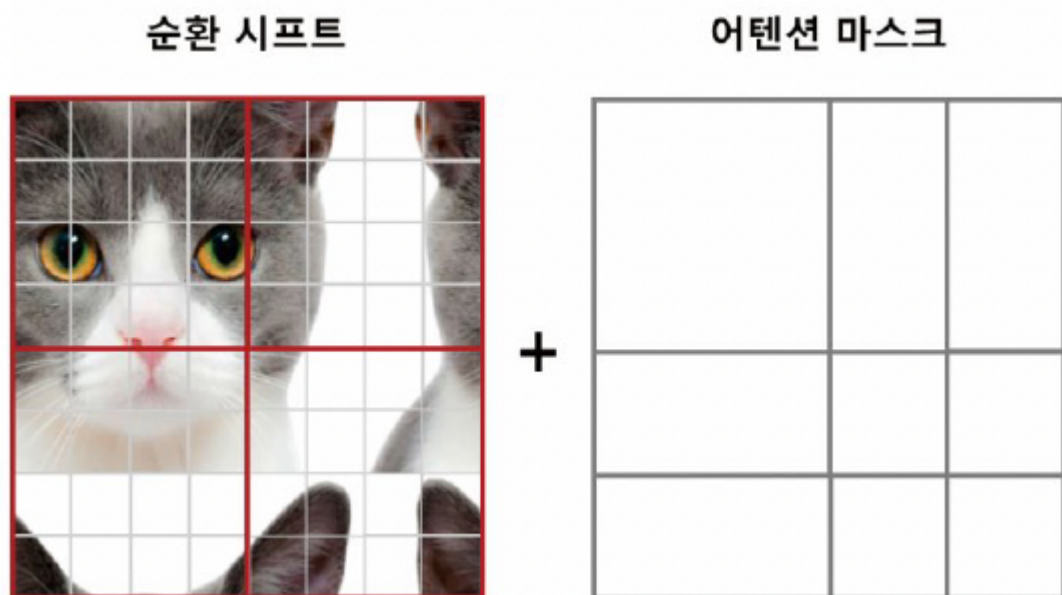


(1) 순환 시프트 (Cyclic Shift)

- 윈도우를 이동시켜 인접 영역 간의 정보 교환 수행
- 어텐션 마스크(Attention Mask)를 함께 사용해
이동된 윈도우 간 불필요한 연산을 차단

(2) 어텐션 마스크 (Attention Mask)

- 특정 윈도우 간 어텐션 계산을 제한하는 역할
- 효율적인 연산 수행과 메모리 절약 가능



ViT vs Swin Transformer 비교

구분	ViT	Swin Transformer
분류 헤드	[CLS] 토큰 사용	각 윈도우의 평균값 사용
로컬 윈도우	X	O
상대적 위치 편향	X	O
패치 크기	동일	계층별 다양
구조	단일 해상도	계층적 구조
특징	전역적 학습	지역+전역 모두 학습 가능

Swin Transformer의 장점

- 지역 정보(Local) + 전역 정보(Global) 모두 학습
- 계산량 효율적
- 다양한 해상도 처리 가능
- ViT보다 높은 정확도, 적은 파라미터

Swin Transformer 내부 블록 구성

- 각 블록은 다음 순서로 수행됨:
 1. 계층 정규화 (Layer Norm)
 2. W-MSA (윈도우 내 어텐션)
 3. 계층 정규화
 4. SW-MSA (이동된 윈도우 어텐션)
 5. MLP (피드포워드 신경망)

상대적 위치 편향 (Relative Position Bias)

(1) 개념

- Self-Attention은 쿼리(Q)와 키(K)의 내적을 통해 관계를 계산하지만, 위치 정보를 직접 고려하지 못함.
- Swin Transformer는 이를 보완하기 위해 상대적 위치 편향(Relative Position Bias)을 추가.

(2) 작동 원리

- 각 패치 간의 상대적인 거리(위치 차이)를 X축, Y축 기준으로 계산.
- 패치가 같은 위치 → 거리 0
- 오른쪽 / 아래로 이동 → +1
- 왼쪽 / 위로 이동 → -1
- 이를 통해 패치 간의 상대적 관계를 수치로 표현.

(3) 상대적 위치 행렬 계산 단계

좌표 생성

```
coords_h = torch.arange(window_size)
coords_w = torch.arange(window_size)
```

→ 각 축의 위치를 나타내는 0~window_size-1 값 생성.

2D 좌표 결합

```
coords = torch.stack(torch.meshgrid([coords_h, coords_w], indexing="ij"))
```

→ 각 패치의 (X, Y) 좌표쌍 생성.

상대 좌표 계산

```
relative_coords = coords[:, :, None] - coords[:, None, :]
```

→ 각 패치쌍 간 거리 차이 계산.

→ (X, Y) 각각에 대해 상대적 거리 행렬 생성.

편향 인덱스 계산

```
x_coords = relative_coords[0] + window_size - 1
y_coords = relative_coords[1] + window_size - 1
x_coords * (2 * window_size - 1) + y_coords
```

→ 상대적 위치를 고유 인덱스로 변환.

편향 테이블 생성

```
relative_position_bias_table = torch.zeros((2*window_size-1)**2, num_head)
```

s)

→ 각 상대 거리(예: 0~8)에 대응되는 bias 값 저장 테이블.

최종 편향 적용

```
relative_position_bias = relative_position_bias_table[relative_position_index.  
view(-1)].view(...)
```

→ attention score 계산 시 bias (B)로 더함.

(4) Self-Attention 수식 반영

[

$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}} + B\right)V$

]

Swin Transformer 모델 구조

(1) 기본 구조

Swin Transformer는 크게 두 부분으로 구성됨:

1. swin: 입력 특징을 계층적으로 변환 (feature extractor)
2. classifier: 최종 분류 수행

Patch Embedding (패치 임베딩)

(1) 역할

- 입력 이미지를 4×4 크기의 패치로 분할 후
Conv2D 연산을 통해 각 패치를 벡터로 변환.

(2) 출력 형태

입력 크기: [3, 224, 224]

→ 패치 수: $(224 / 4)^2 = 56 \times 56 = 3136$

→ 출력 크기: [batch, 3136, 96]

ViT의 [197, 768]과 비교하면, Swin은 더 세밀하고 지역적인 정보 보존.

Swin Transformer의 Encoder 구조

(1) 계층(Stage) 구성

Stage	입력 크기	패치 수	채널 수	특징
Stage 1	56×56	3136	96	기본 윈도우 (7×7)
Stage 2	28×28	784	192	Patch Merging 수행
Stage 3	14×14	196	384	깊은 계층
Stage 4	7×7	49	768	전역적 특징 추출

SwinLayer 블록 구조

각 Stage는 여러 SwinLayer로 구성되어 있으며,

각 SwinLayer는 다음 순서로 실행됨:

LayerNorm → (W-MSA 또는 SW-MSA) → LayerNorm → MLP

MLP는 2개의 선형 계층과 GELU 활성화함수로 구성됨.

W-MSA & SW-MSA

(1) W-MSA (Window-based Multi-Head Self-Attention)

- 하나의 로컬 윈도우 내에서만 어텐션 수행.
- 계산량 ↓, 지역적 특징 학습 ↑
- 하지만 윈도우 간 정보 교환은 불가능.

(2) SW-MSA (Shifted Window MSA)

- 윈도우를 반 칸 이동(Shift)시켜, 인접 윈도우 간 정보도 학습.
- Attention Mask를 통해 불필요한 연산 방지.

(3) 두 방식의 관계

- Stage 내에서 W-MSA → SW-MSA 순서로 수행.
- 즉, 하나의 블록이 W-MSA, 다음 블록이 SW-MSA.

Patch Merging (패치 병합)

(1) 역할

- 인접한 패치 2×2 를 하나로 병합 → 해상도 절반으로 감소.
- 채널 수는 2배 증가.

(2) 출력 예시

단계	입력 크기	출력 크기
입력	[32, 3136, 96]	
출력	[32, 784, 192]	

전체 Swin Transformer 파이프라인

입력

- [batch, 3, 224, 224]

패치 분할 (4×4)

- [batch, 3136, 96]

Stage 1~4 통과 (Patch Merging 포함)

- [N, 3136, 96] → [N, 784, 192] → [N, 196, 384] → [N, 49, 768]

Layer Normalization + Pooling

- [N, 49, 768] → [N, 768]

Classifier (Linear Layer)

- [N, 768] → [N, num_classes]

학습 과정

(1) 사전학습 모델 불러오기

```
model = SwinForImageClassification.from_pretrained(  
    "microsoft/swin-tiny-patch4-window7-224",  
    num_labels=len(classes),  
    id2label=...,  
    label2id=...,
```

```
ignore_mismatched_sizes=True
)
```

(2) 이미지 프로세서

```
image_processor = AutoImageProcessor.from_pretrained(
    "microsoft/swin-tiny-patch4-window7-224"
)
```

(3) 전처리

- Resize(224×224), Normalize(mean/std), Tensor 변환

(4) 학습

- Trainer를 이용해 3 Epoch 학습
- Optimizer와 Scheduler는 자동 설정

(5) 평가

- Macro F1 score 및 Confusion Matrix로 모델 성능 분석
- 약 92% 정확도 달성

(6) 학습 결과

Epoch	Training Loss	Validation Loss	F1 Score
1	0.3720	0.3046	0.8985
2	0.2579	0.2688	0.9138
3	0.2607	0.2416	0.9159

- F1 성능은 ViT와 거의 동일하지만, Validation Loss는 0.4357 → 0.2416으로 크게 감소하여 더 안정적인 학습 성능을 보임.

- 최종 분류 정확도는 약 91.6%

(7) 혼동 행렬 결과 (Confusion Matrix)

- Swin Transformer는 ViT보다 '셔츠(Shirt)'를 '코트(Coat)'로 잘못 분류하는 문제가 완화됨.
- 학습 및 추론 속도도 ViT보다 빠름.

→ 즉, Swin Transformer는 더 효율적이며 정교한 시각적 구분 능력을 보여줌.

CvT (Convolutional Vision Transformer)

(1) 기본 개념

- CvT는 합성곱 신경망(CNN)의 계층적 구조(Hierarchical Architecture)를 비전 트랜스포머(ViT)에 결합한 모델
- 쉽게 말해, CNN의 "지역적 특징 학습력(Local Feature)"과 Transformer의 "전역적 문맥 학습력(Global Feature)"을 모두 활용하는 하이브리드 모델이다.

(2) Swin Transformer와의 차이

구분	Swin Transformer	CvT
윈도우 방식	지역 윈도우 단위로 나눔	합성곱 기반 지역 특징 추출
구조적 특징	패치 단위 계층 구조	CNN의 계층적 구조 결합
학습 특징	지역+전역 윈도우 이동으로 특징 결합	합성곱 연산으로 지역→전역 특징 추출

(3) 계층적 특징 구조 (Hierarchical Feature Extraction)

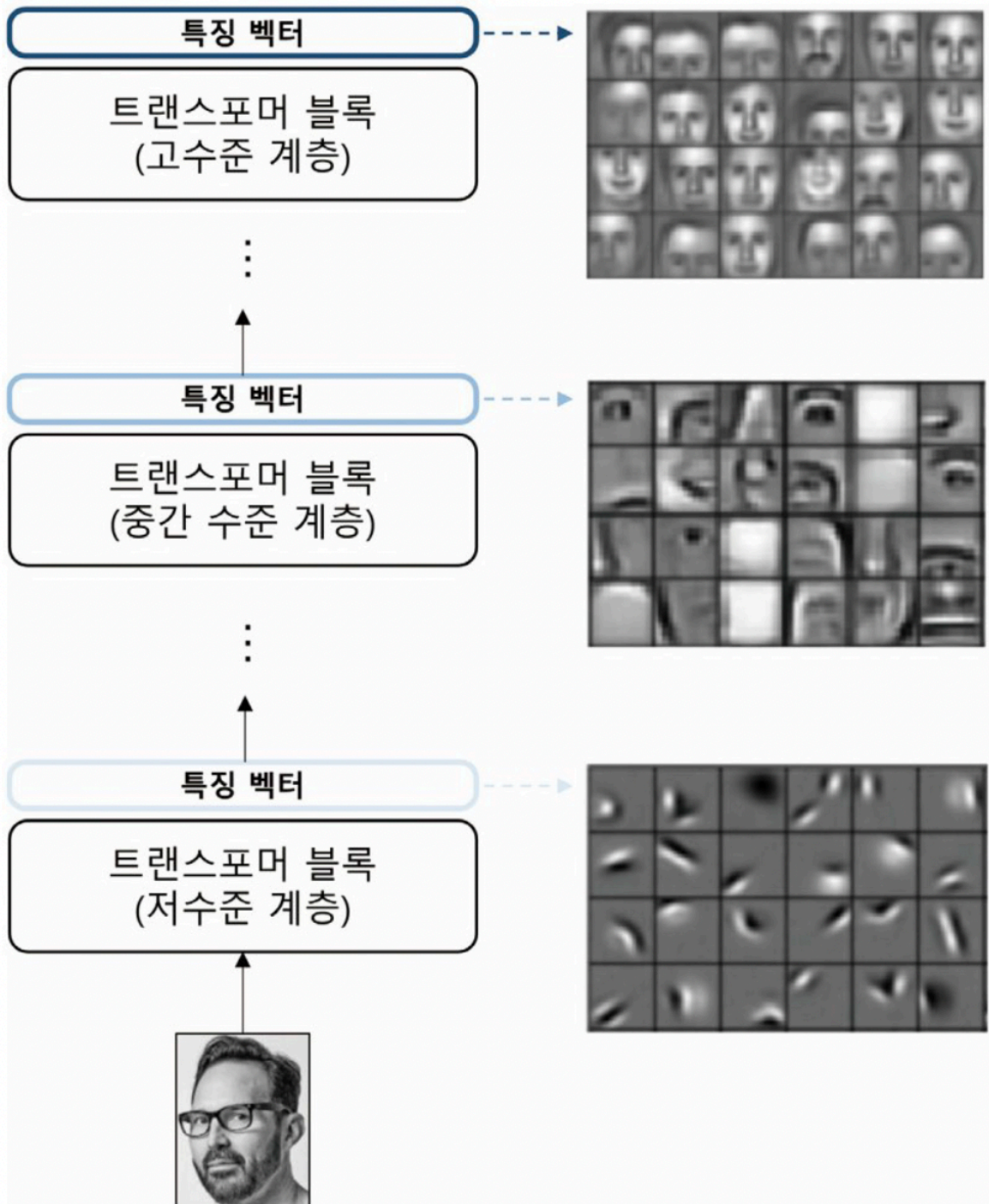


그림 10.14 CVT 모델의 계층적 특징 구조 예시

CVT는 이미지를 저수준 → 중간 수준 → 고수준 특징으로 점진적으로 학습함.

계층	의미	예시
저수준 (Low-level)	선, 점, 모서리 등 단순한 형태	선, 윤곽선
중간 수준 (Mid-level)	눈, 코, 귀 등 부분적 형태	얼굴 일부

계층	의미	예시
고수준 (High-level)	전체 구조나 패턴	얼굴 전체

ViT vs CvT 비교

항목	ViT	CvT
위치 임베딩	필요 (절대적 위치 정보 추가)	불필요 (합성곱이 위치 정보 내포)
패치 임베딩	겹치지 않음(non-overlapping)	겹치는 합성곱 임베딩(overlapping)
어텐션 임베딩	선형 임베딩	합성곱 임베딩
구조	비계층적 (flat)	계층적 (hierarchical)

→ CvT는 ViT보다 더 적은 매개변수로 비슷하거나 더 좋은 성능을 낼 수 있음.

합성곱 토큰 임베딩 (Convolutional Token Embedding)

(1) 개념

Convolutional Token Embedding은

기존의 단순한 패치 분할 대신 2D 합성곱 연산을 통해 임베딩을 생성하는 방식이다.

- ViT: 이미지를 단순히 패치로 자른 후 → 선형 변환(Linear Embedding)
- CvT: 합성곱 연산(Conv2D)을 적용해 겹치는 영역의 정보도 포함

즉, CvT는 합성곱으로 토큰(패치)을 더 풍부하게 표현한다.

(2) 시각적 개념

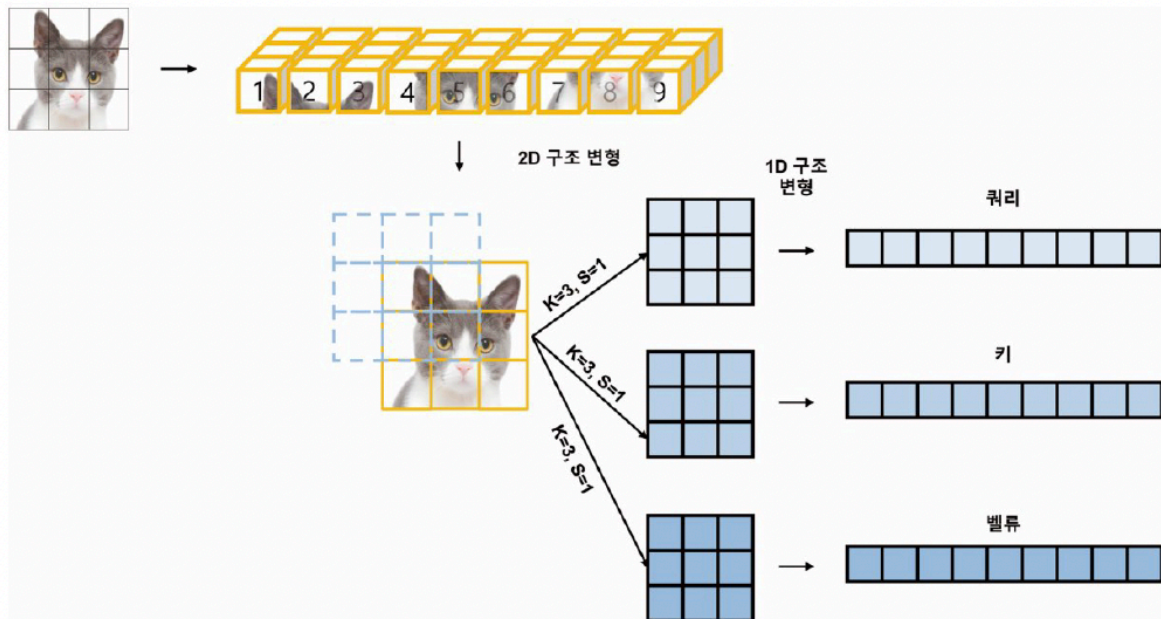


그림 10.15 2D 합성곱 기반의 어텐션 임베딩

- 이미지에 3×3 커널을 적용해 9개의 패치를 생성
- 각 패치에서 쿼리(Q), 키(K), 값(V)을 합성곱으로 계산
→ 기존 1D 구조가 아닌 2D 구조 기반 어텐션
- 이렇게 하면 지역적 특징 보존 + 시퀀스 길이 단축 + 토큰 정보량 증가

어텐션에 대한 합성곱 임베딩 (Convolutional Projection for Attention)

(1) 개념

Convolutional Projection for Attention은

기존의 Q, K, V 연산에 합성곱 필터를 적용해 연산 효율을 높이는 방법이다.

- 기존 Transformer:
Q, K, V를 동일한 차원으로 선형 변환 후 dot-product 계산
- CvT:
Q, K, V를 합성곱으로 생성하여 공간적 구조 보존 + 차원 축소로 연산량 감소

2D 합성곱 기반 축소형 어텐션 임베딩

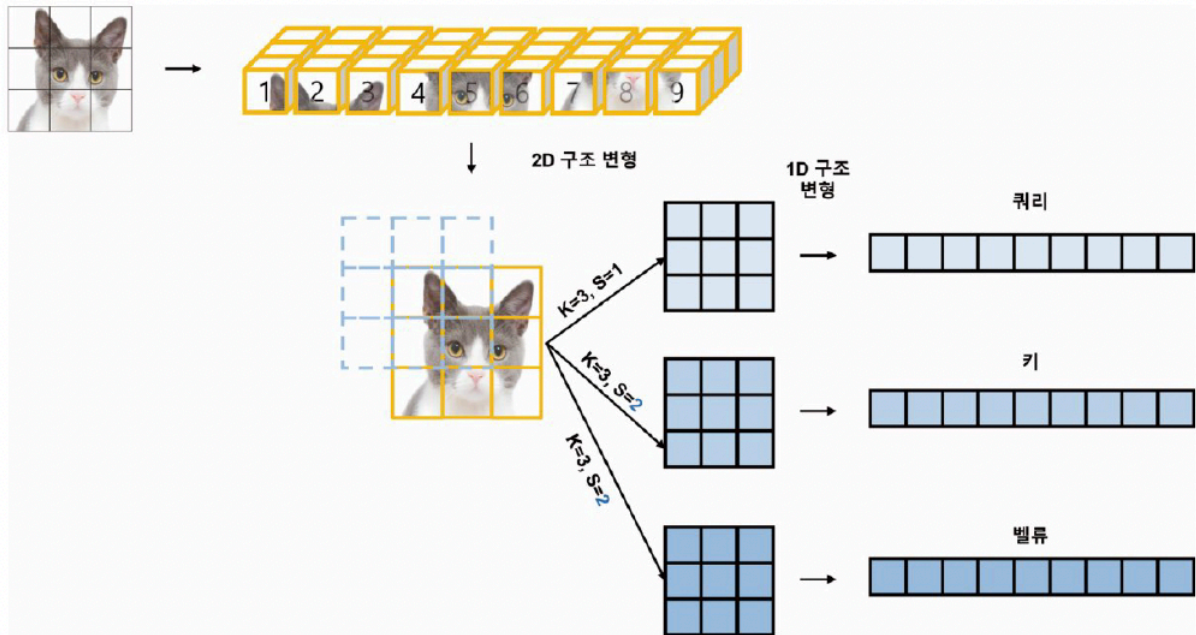


그림 10.16 2D 합성곱 기반의 축소된 어텐션 임베딩

단계	설명
1	이미지에 3×3 커널, 1 또는 2의 스트라이드(S)를 적용하여 패치 생성
2	쿼리(Q), 키(K), 값(V)를 각기 다른 합성곱 필터로 계산
3	$Q \times K^T$ 로 어텐션 맵 생성
4	어텐션 맵 $\times$ 값(V) $\rightarrow$ 최종 셀프 어텐션 벡터 출력

→ 이 과정에서 패치 개수는 줄고 임베딩 차원은 늘어남 → 계산 효율 증가.

CvT의 모델 설계 의의

- 패치 크기를 점점 줄이고, 임베딩 차원을 점점 늘리는 방식으로 구조 단순화.
- 계산해야 하는 패치 수가 줄어들어 연산량 감소.
- 대신 벡터의 차원을 증가시켜 풍부한 표현력 유지.
- 결과적으로 더 빠르고 안정적인 학습 + 더 깊은 계층 구조 학습 가능.

CvT와 ViT, Swin Transformer의 차이점

구분	ViT	Swin Transformer	CvT
임베딩 방식	선형 임베딩	패치 파티션 + 선형 임베딩	합성곱 임베딩(Convolutional Embedding)

구분	ViT	Swin Transformer	CvT
구조	비계층적	계층적 (윈도우 기반)	계층적 (합성곱 계층 기반)
패치 크기	고정	단계별 변화	단계별 변화
지역/전역 특징	전역 중심	지역 중심	지역 + 전역 특징 통합

CvT의 주요 구성 요소

(1) 합성곱 토큰 임베딩 (Convolutional Token Embedding)

- 2D 합성곱 연산을 통해 이미지의 공간 정보를 유지하며 토큰화합니다.
- 패치를 잘라내는 ViT와 달리, 중첩되는(overlapping) 패치를 만들어 세밀한 특징을 더 잘 포착합니다.
- 각 패치는 이후 쿼리(Q), 키(K), 값(V)로 변환되어 셀프 어텐션 연산에 사용됩니다.

(2) 어텐션에 대한 합성곱 임베딩 (Convolutional Projection for Attention)

- Q, K, V에 대해 각각 3×3 합성곱 연산을 적용하여 공간적인 정보 보존.
- 계산 복잡도를 줄이기 위해 ****Squeezed Projection (축소 투영)****을 사용합니다.
- 결과적으로 적은 연산량으로 더 풍부한 표현을 얻습니다.

CvT의 계층 구조 (Hierarchical Architecture)

- CvT는 총 3개의 Stage로 구성됩니다.
 - 각 Stage는 CvtEmbedding + CvtLayer로 구성.
 - 각 Stage마다 이미지 크기는 줄고, 채널(특징 수)은 증가합니다.
 - 예:
 - Stage 1 → 56×56, 64채널
 - Stage 2 → 28×28, 192채널
 - Stage 3 → 14×14, 384채널
 - 즉, 이미지가 점점 압축되면서 더 높은 수준의 특징을 학습합니다.

CvT 학습 결과 (FashionMNIST 실험)

Epoch	Training Loss	Validation Loss	F1 Score
1	0.7737	0.3269	0.8914

Epoch	Training Loss	Validation Loss	F1 Score
2	0.7050	0.2596	0.9172
3	0.6438	0.2460	0.9173

- 최종적으로 91.3%의 정확도, F1 = 0.9173을 달성.
- Swin Transformer(0.9159)보다 약간 높고, ViT(0.9231)와 유사한 수준입니다.

Confusion Matrix 분석

- CvT는 특히 '셔츠(Shirt)' 카테고리의 인식률이 ViT보다 향상되었습니다.
- '코트(Coat)'와 '셔츠(Shirt)' 간 혼동 문제도 일부 해결되었습니다.
- Swin과 비교하면 유사한 수준의 성능을 가지며, 학습 속도는 ViT보다 효율적입니다.

모델 크기 및 성능 비교

모델	크기	F1 점수	초당 처리 속도
ViT	327.325 MB	0.9231	29.636 eval/s
Swin Transformer	105.227 MB	0.9159	58.048 eval/s
CvT	120.791 MB	0.9173	44.778 eval/s

- ViT: 가장 높은 성능 (F1)
- Swin: 가장 가벼움, 빠른 처리 속도
- CvT: 균형 잡힌 모델 — 높은 정확도 + CNN의 지역 특징 활용